AI-Driven File System Performance Optimization

Course Number: SSZG628T

Dissertation / Project /Project Work Done by:

Student Name: Mayur Raj Bharati

BITS ID: 2024MT12012

Degree Program: Masters in Technology (M.Tech) in Software Systems

Research Area: Operating Systems

Dissertation / Project Work carried out at:

Veritas Technologies, Pune

Submitted in partial fulfilment of **Masters of Technology** degree programme

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
VIDYA VIHAR, PILANI, RAJASTHAN - 333031.

March, 2026

Contents

1. Abstract 3
2. Introduction 3
3. Dataset Description 4
4. Data Preprocessing 4
5. Methodology 5
6. Model Comparison 8
7. Future Work 8
8. Conclusion 9

1. Abstract

Filesystem performance is highly dependent on multiple interacting configuration parameters such as disk block size, read-ahead settings, and I/O scheduler policies. Identifying optimal parameter combinations manually is challenging due to the large configuration space and complex workload interactions. This project explores the use of machine learning techniques to model filesystem performance and predict the optimal set of configuration of these tunable parameters, based on various performance metrics like bandwidth, latency and IOPS. A dataset of approximately 10,000 filesystem experiments was collected and used to train multiple ensemble-based regression models. Several tree-based machine learning algorithms were evaluated and compared using standard performance metrics. Experimental results demonstrate that gradient boosting based models provide superior predictive accuracy for this problem. The project establishes a foundation for building an automated filesystem configuration optimization system.

2. Introduction

Modern file systems expose a wide range of tunable parameters that significantly affect filesystem performance. Parameters such as block size, read-ahead size, commit interval, journaling mode, and I/O scheduling policies determine how efficiently storage devices handle different workloads.

In real-world deployments, system administrators often rely on default settings or manual experimentation to tune these parameters. However, as the number of configurable parameters increases, identifying optimal combinations becomes increasingly difficult. This requires very skilled personnel and continuous monitoring as the IO workload is very heterogenous in nature.

Machine learning provides a data-driven approach to model the relationship between filesystem configurations and performance outcomes. By training predictive models on experimental data, it becomes possible to estimate the best set of these parameters for optimum performance.

In addition to performance optimization, practical filesystem deployment must also consider reliability and security constraints. Certain configuration parameters, particularly journaling modes, directly influence data consistency guarantees and crash recovery behavior. Therefore, this project introduces a security-aware optimization layer in which a user-specified security level constrains the configuration space explored by the optimizer. This ensures that recommended configurations not only maximize performance metrics but also comply with required security and reliability policies.

The goal of this project is to develop a machine learning based framework capable of predicting filesystem performance metrics and assisting in adaptive configuration optimization for changing IO workloads.

The primary objectives of this project are:

- Collect and prepare a dataset of filesystem performance experiments.
- Apply machine learning techniques to predict performance metrics.
- Compare multiple regression models for prediction accuracy.

- Identify the most effective model for filesystem performance prediction.

3. Dataset Description

The dataset used in this project contains approximately 10,000 filesystem experiment records. Each record represents a specific filesystem configuration tested under a particular workload. The dataset includes the following filesystem parameters:

- workload type
- block size
- journal mode
- io scheduler
- read ahead sizes
- barriers
- noatime options
- commit intervals
- io engines

These parameters define how the filesystem handles I/O operations.
Each configuration produces three primary performance metrics:

- latency
- bandwidth
- IOPS

Dataset Augmentation: The current dataset has ~10k rows which covers almost 90% of the full configuration space.
Since this dataset covers almost 90% of the configuration space, hence it is not required to perform any sort of dataset augmentation for our purpose.

4. Data Preprocessing

To improve model performance and capture relationships between parameters, additional derived features were created:

- **Read-ahead ratio:** This feature captures the relationship between block size and read-ahead size.

  read_ahead_ratio = read_ahead_sizes / block_size
- **Commit density:** This feature captures the relative commit frequency compared to block size.

  commit_density = commit_intervals / block_size
- **Workload pattern classification:** Workloads were categorized into two groups:
  - **Sequential workloads**
  - **Random workloads**

This classification allows models to learn different behavior patterns for each workload type.

- **Scheduler classification:** IO schedulers were grouped based on their primary optimization objective:

- **Latency oriented schedulers**
- **Fairness oriented schedulers**

These engineered features improve the model's ability to capture system-level relationships among parameters.

5. Methodology

The problem is formulated as a multi-output regression task where the model predicts multiple performance metrics simultaneously.
Various pre-processing steps were applied:

- Categorical features encoded using one-hot encoding
- Numeric engineered features passed directly to the model
- Dataset split into training and testing set
- ***k*** fold cross validation was used to obtain a more reliable estimate of model performance. In this method, the training dataset is divided into k smaller subsets (folds). The model is trained on k-1 folds and validated on the remaining fold. This process is repeated multiple times so that each fold serves as a validation set once.

Models Evaluated:

- ***Random Forest*:** Random Forest constructs multiple decision trees using bootstrap sampling and aggregates their predictions.
- ***Extra Trees***: Extra Trees introduces additional randomness by selecting split thresholds randomly during tree construction**.**
- ***Gradient Boosting Model:*** Two gradient boosting frameworks were evaluated:
  - ***XGBoost***
  - ***LightGBM***

These models build trees sequentially, where each tree attempts to correct errors made by the previous tree.

Model Evaluation Metrics:
Model performance was evaluated using two primary metrics:

- ***R-squared (R^2):*** The R^2 score measures how well the model explains the variance in the dataset. Higher values indicate better predictive performance.
- ***Root Mean Squared Error (RMSE):*** RMSE measures the average magnitude of prediction errors. Lower values indicate better model accuracy.

Security aware configuration constraints:

In practical storage deployments, filesystem configuration decisions must also respect security requirements. Certain configurations, particularly journaling modes, may provide stronger consistency guarantees but can introduce additional performance overhead. To address this, the system incorporates a ***user-controlled security level input***.

The user specifies a desired security level (***for example: low, medium, or high***). This input acts

as a constraint layer that restricts the set of filesystem parameters considered during optimization. In this project, the constraint primarily affects journaling modes, ensuring that only configurations compatible with the chosen security level are explored. This design allows the optimization system to balance performance and reliability requirements, preventing the optimizer from recommending configurations that violate the desired security policy.

Configuration optimization using Bayesian optimization:

After the predictive model is trained and the security constraints are applied, an optimization step is used to search for the best filesystem configuration parameters.

Instead of performing an exhaustive search over the large configuration space, Bayesian Optimization is used as an efficient strategy to explore promising configurations. Bayesian optimization builds a probabilistic model of the objective function and iteratively selects configurations that are expected to improve performance.

Using the trained machine learning model as a surrogate performance predictor, the optimizer evaluates candidate configurations that satisfy the security-level constraints and identifies parameter combinations that gives optimum performance.

The final output of this step is ***a recommended set of filesystem configurations*** predicted to provide optimal performance for a given workload while respecting the user-specified security requirements.

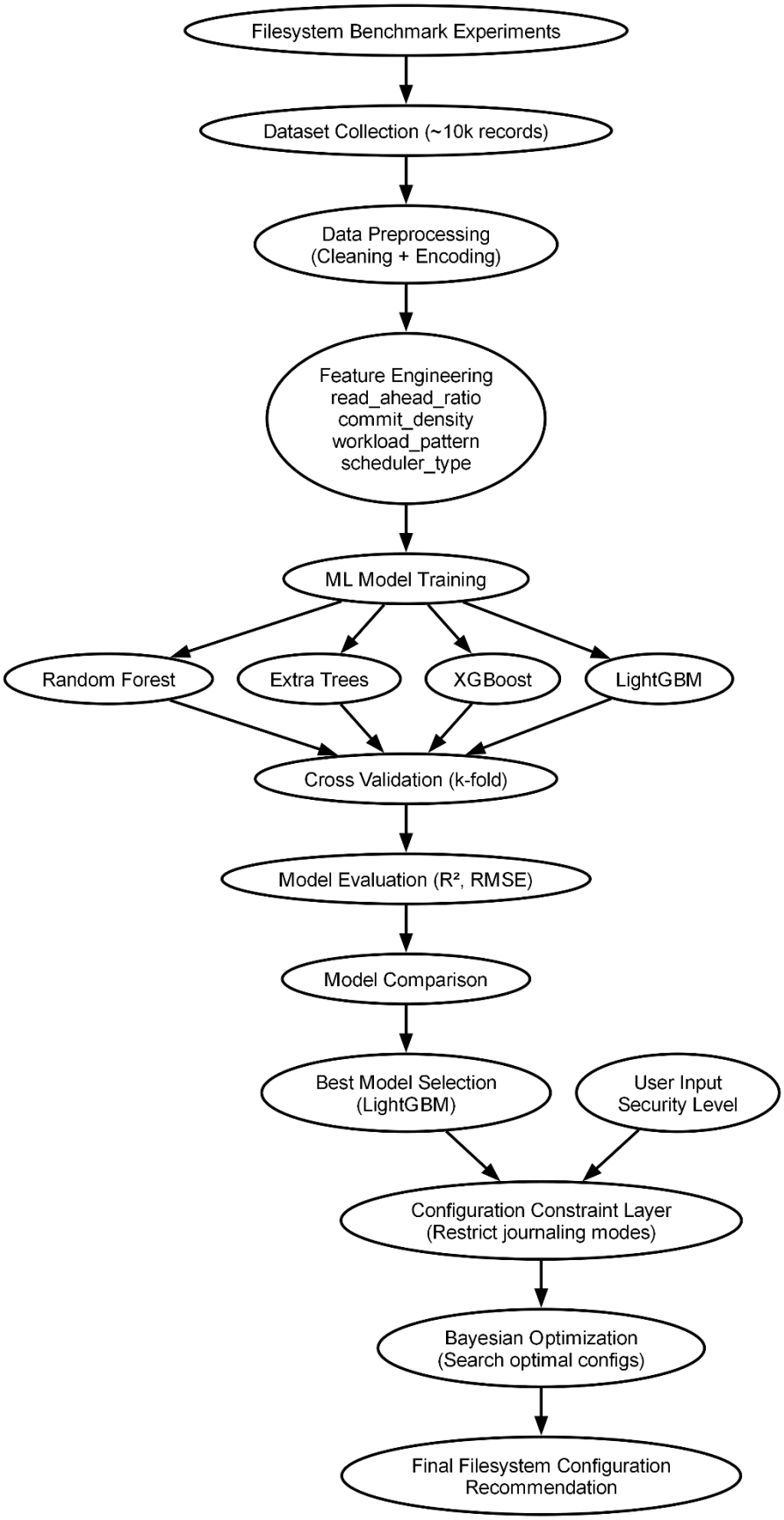

Figure: Architectural diagram

6. Model Comparison

All models were trained using identical preprocessing pipelines and evaluated on the same test dataset.

Model	Approximate R^2 score
Random Forest	**0.809**
Extra Trees	**0.539**
XGBoost	**0.794**
LightGBM	**0.829**

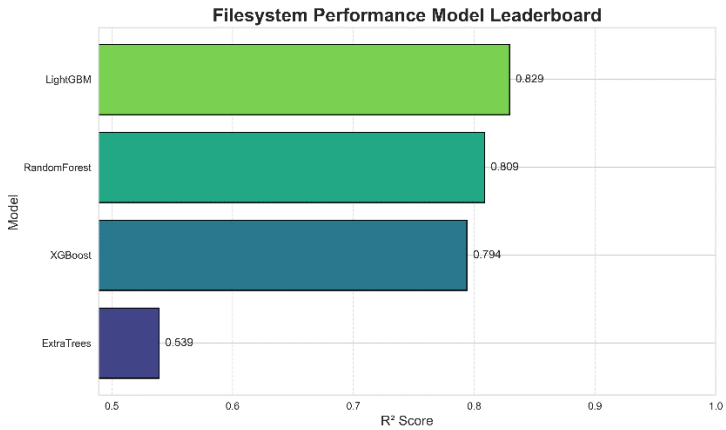

Figure: Comparison of R^2 scores among various models

The results indicate that the gradient boosting models outperform traditional ensemble methods for this dataset.

Among all evaluated models, ***LightGBM*** achieved the highest predictive accuracy with the best overall R^2 score across latency, bandwidth, and IOPS metrics, and was therefore selected as the primary model for further development.

7. Future Work

Several improvements can be made for the next phase of this project:

- **Hyperparameter optimization:** Model performance can be further improved by tuning hyperparameters of the ***LightGBM*** model using automated search techniques such as randomized search.
- **Pareto based configuration optimization:** Future work will implement Pareto optimization techniques like pruning to identify configurations that achieve optimal trade-offs between these metrics.
- **Transfer Learning:** Since previous research was done in this field, we can explore and adapt transfer learning techniques for model training purpose.
- **Add another feature:** We can add another feature in our dataset viz, hardware used as the kind of hardware used during IO workloads influences performance significantly.
- **Integrate this model with a kernel module:** We can integrate this optimizer tool with a

kernel module thereby providing the luxury to plug in this kernel module and perform continuous monitoring for the filesystem in question.

- **Dynamic workload analysis:** Currently the model takes in input as the workload type and the security level the user wants, however in the future we would want to analyze the workload dynamically instead of manual user input.

8. Conclusion

This project explored the use of machine learning techniques to model and optimize filesystem performance based on configuration parameters and workload characteristics. A dataset containing approximately 10,000 filesystem experiment records was used to train and evaluate several ensemble-based regression models. The models were designed to predict key performance metrics, including latency, bandwidth, and IOPS, from a combination of filesystem configuration parameters.

Several tree-based ensemble learning algorithms were evaluated, including Random Forest, Extra Trees, XGBoost, and LightGBM. Cross-validation was applied to ensure reliable model evaluation and reduce the risk of overfitting. Experimental results indicated that gradient boosting methods provide superior predictive performance for this problem domain, with LightGBM achieving the highest overall accuracy among the evaluated models.

Beyond predictive modeling, the system design incorporates a security-aware optimization framework. A user-specified security level acts as a constraint layer that restricts the set of valid filesystem configurations, particularly influencing journaling mode selection. This ensures that performance optimization does not violate reliability or security requirements.

Using the trained machine learning model as a surrogate predictor, Bayesian optimization is employed to efficiently search the configuration space and identify filesystem parameter combinations that optimize performance objectives while respecting security constraints.

Overall, the proposed approach demonstrates that machine learning can effectively capture complex relationships between filesystem parameters and performance metrics. By combining predictive modeling with constraint-aware optimization, the system provides a promising foundation for automated filesystem configuration tuning that balances performance, reliability, and security considerations.